AI LAB ASSIGNMENT – 6

ADMISSION NO : U24AI040

Q1 Use the city map given in Q1 of assignment 1 and implement the greedy best first search and A* algorithm to reach Boston city from Chicago and compare your results in terms of the number of cities explored before reaching your destination. Consider the heuristic values as given below,

City h(n) to Boston

(miles)

Boston 0

Providence 50

Portland 107

New York 215

Philadelphia 270

Baltimore 360

Syracuse 260

Buffalo 400

Pittsburgh 470

Cleveland 550

Columbus 640

Detroit 610

Indianapolis 780

Chicago 860

ANS:

GREEDY BEST FIRST SEARCH:

```
city_map={
    0 : "Chicago",
    1 : "Detroit",
    2 : "Cleveland",
    3 : "Indianapolis",
```

```
    4 : "Columbus",
    5 : "Buffalo",
    6 : "Pittsburgh",
    7 : "Syracuse",
    8 : "Philadelphia",
    9 : "Baltimore",
    10 : "New York",
    11 : "Providence",
    12 : "Boston",
    13 : "Portland"
}

heuristic=[860,610,550,780,640,400,470,260,270,360,215,50,0,107]


INT_MAX = 10000000

class PQ:
    def __init__(self):
        self.q=[]

    def isEmpty(self):
        return len(self.q)==0
    def right(self,i):
        return 2*i + 2
    def left(self,i):
        return 2*i + 1
    def parent(self,i):
        return (i-1)//2
    def insert(self, val):
        self.q.append(val)
```

```
    i = len(self.q) - 1

    while i>0:
        p=self.parent(i)
        if self.q[p].h > self.q[i].h:
            self.q[p], self.q[i] = self.q[i], self.q[p]
            i=p
        else:
            break

def pop(self):
    if self.isEmpty():
        return None
    if len(self.q)==1:
        return self.q.pop()
    r=self.q[0]
    self.q[0]=self.q.pop()
    self.heapify(0)
    return r
def heapify(self,i):
    l=self.left(i)
    r=self.right(i)
    mi=i
    if l<len(self.q) and self.q[l].h<self.q[mi].h:
        mi=l
    if r<len(self.q) and self.q[r].h<self.q[mi].h:
        mi=r
    if mi!=i:
        self.q[i],self.q[mi]=self.q[mi],self.q[i]
        self.heapify(mi)
```

```
class Node:
    def __init__(self,state,parent,action):
        self.state = state
        self.parent = parent
        self.action = action
        self.h = heuristic[state]

class Problem:
    def __init__(self,start,goal,adj):
        self.initial_state=start
        self.goal_state=goal
        self.mat=adj

    def INITIAL(self):
        return self.initial_state

    def Is_Goal(self,state):
        return state==self.goal_state

    def ACTIONS(self,s):
        res=[]
        for node in range(len(self.mat)):
            if self.mat[s][node]!=INT_MAX and node!=s:
                res.append(node)
        return res
    def RESULT(self,state,action):
        return action

    def ACTION_COST(self,s,a,S):
        return self.mat[s][S]
```

```
def Expand(problem,node):
    res = []
    s = node.state

    for action in problem.ACTIONS(s):
        new = problem.RESULT(s, action)
        child = Node(state=new,parent=node,action=action)
        res.append(child)

    return res

def Greedy(problem):
    frontier=PQ()
    start=Node(state=problem.INITIAL(),parent=None,action=None)

    frontier.insert(start)
    reached={problem.INITIAL() : start}
    count=0

    while not frontier.isEmpty():
        node=frontier.pop()
        count+=1
        if problem.Is_Goal(node.state):
            print("Cities Visited",count)
            return node
        for child in Expand(problem,node):
            s=child.state
            if s not in reached :
                reached[s] = child
                frontier.insert(child)
```

```
    return None

def get_best_path(goal_node):
    path = []
    node = goal_node

    while node is not None:
        path.append(node.state)
        node = node.parent

    path.reverse()
    return path
def cost(path,adj):
    c=0
    for i in range(len(path)-1):
        c+=adj[path[i]][path[i+1]]
    return c
def addEdge(graph, u, v, d):
    graph[u][v] = d
    graph[v][u] = d

def main():
    n = 14
    adj = []

    for i in range(n):
        row = []
        for j in range(n):
            row.append(INT_MAX)
        adj.append(row)
    for i in range(n):
```

```
    adj[i][i] = 0

addEdge(adj, 0, 1, 283)
addEdge(adj, 0, 2, 345)
addEdge(adj, 0, 3, 182)
addEdge(adj, 3, 4, 176)
addEdge(adj, 4, 2, 144)
addEdge(adj, 4, 6, 185)
addEdge(adj, 2, 1, 169)
addEdge(adj, 2, 5, 189)
addEdge(adj, 2, 6, 134)
addEdge(adj, 1, 5, 256)
addEdge(adj, 5, 7, 150)
addEdge(adj, 5, 6, 215)
addEdge(adj, 6, 8, 305)
addEdge(adj, 6, 9, 247)
addEdge(adj, 9, 8, 101)
addEdge(adj, 8, 10, 97)
addEdge(adj, 10, 11, 181)
addEdge(adj, 10, 12, 215)
addEdge(adj, 12, 11, 50)
addEdge(adj, 12, 13, 107)
addEdge(adj, 7, 12, 312)
addEdge(adj, 7, 8, 253)
addEdge(adj, 7, 10, 254)

problem = Problem(0, 12, adj)
result = Greedy(problem)

if result is not None:
    path = get_best_path(result)
```

```
        print("Best path :")
        for i in path:
            print(city_map[i],sep=" ")

        path_cost=cost(path,adj)
        print(path_cost)
    else:
        print("No path found")



main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\VIVEK\Desktop\AI_ Lab> python -u "c:\Users\VIVEK\Desktop\AI_ Lab\Lab Assignment 6\que1.py"
Cities Visited 5
Best path :
Chicago
Cleveland
Buffalo
Syracuse
Boston
996
PS C:\Users\VIVEK\Desktop\AI_ Lab>
```

A* SEARCH:

```
city_map={
    0 : "Chicago",
    1 : "Detroit",
    2 : "Cleveland",
    3 : "Indianapolis",
    4 : "Columbus",
    5 : "Buffalo",
    6 : "Pittsburgh",
    7 : "Syracuse",
    8 : "Philadelphia",
```

```
    9 : "Baltimore",
    10 : "New York",
    11 : "Providence",
    12 : "Boston",
    13 : "Portland"
}

heuristic=[860,610,550,780,640,400,470,260,270,360,215,50,0,107]

INT_MAX = 10000000

class PQ:
    def __init__(self):
        self.q=[]

    def isEmpty(self):
        return len(self.q)==0
    def right(self,i):
        return 2*i + 2
    def left(self,i):
        return 2*i + 1
    def parent(self,i):
        return (i-1)//2
    def insert(self, val):
        self.q.append(val)
        i = len(self.q) - 1

        while i>0:
            p=self.parent(i)
            if self.q[p].f > self.q[i].f:
                self.q[p], self.q[i] = self.q[i], self.q[p]
```

```
            i=p
        else:
            break

    def pop(self):
        if self.isEmpty():
            return None
        if len(self.q)==1:
            return self.q.pop()
        r=self.q[0]
        self.q[0]=self.q.pop()
        self.heapify(0)
        return r
    def heapify(self,i):
        l=self.left(i)
        r=self.right(i)
        mi=i
        if l<len(self.q) and self.q[l].f<self.q[mi].f:
            mi=l
        if r<len(self.q) and self.q[r].f<self.q[mi].f:
            mi=r
        if mi!=i:
            self.q[i],self.q[mi]=self.q[mi],self.q[i]
            self.heapify(mi)

class Node:
    def __init__(self,state,parent,action,g):
        self.state = state
        self.parent = parent
        self.action = action
        self.g = g
```

```
        self.h = heuristic[state]
        self.f = self.g + self.h

class Problem:
    def __init__(self,start,goal,adj):
        self.initial_state=start
        self.goal_state=goal
        self.mat=adj

    def INITIAL(self):
        return self.initial_state

    def Is_Goal(self,state):
        return state==self.goal_state

    def ACTIONS(self,s):
        res=[]
        for node in range(len(self.mat)):
            if self.mat[s][node]!=INT_MAX and node!=s:
                res.append(node)
        return res
    def RESULT(self,state,action):
        return action

    def ACTION_COST(self,s,S):
        return self.mat[s][S]

def Expand(problem,node):
    res = []
    s = node.state
```

```
    for action in problem.ACTIONS(s):
        new = problem.RESULT(s, action)
        edge_cost = problem.ACTION_COST(s,new)
        child_g = node.g + edge_cost
        child = Node(state=new,parent=node,action=action,g=child_g)
        res.append(child)

    return res

def A_star(problem):
    frontier=PQ()
    start=Node(state=problem.INITIAL(),parent=None,action=None,g=0)

    frontier.insert(start)
    reached={problem.INITIAL() : start}
    count=0
    while not frontier.isEmpty():
        node=frontier.pop()
        count+=1
        if node.g > reached[node.state].g:
            continue
        if problem.Is_Goal(node.state):
            print("Cities Visited",count)
            return node
        for child in Expand(problem,node):
            s=child.state
            if s not in reached or child.g < reached[s].g:
                reached[s] = child
                frontier.insert(child)

    return None
```

```
def get_best_path(goal_node):
    path = []
    node = goal_node

    while node is not None:
        path.append(node.state)
        node = node.parent

    path.reverse()
    return path
def cost(path,adj):
    c=0
    for i in range(len(path)-1):
        c+=adj[path[i]][path[i+1]]
    return c
def addEdge(graph, u, v, d):
    graph[u][v] = d
    graph[v][u] = d

def main():
    n = 14
    adj = []

    for i in range(n):
        row = []
        for j in range(n):
            row.append(INT_MAX)
        adj.append(row)
    for i in range(n):
        adj[i][i] = 0
```

```
addEdge(adj, 0, 1, 283)
addEdge(adj, 0, 2, 345)
addEdge(adj, 0, 3, 182)
addEdge(adj, 3, 4, 176)
addEdge(adj, 4, 2, 144)
addEdge(adj, 4, 6, 185)
addEdge(adj, 2, 1, 169)
addEdge(adj, 2, 5, 189)
addEdge(adj, 2, 6, 134)
addEdge(adj, 1, 5, 256)
addEdge(adj, 5, 7, 150)
addEdge(adj, 5, 6, 215)
addEdge(adj, 6, 8, 305)
addEdge(adj, 6, 9, 247)
addEdge(adj, 9, 8, 101)
addEdge(adj, 8, 10, 97)
addEdge(adj, 10, 11, 181)
addEdge(adj, 10, 12, 215)
addEdge(adj, 12, 11, 50)
addEdge(adj, 12, 13, 107)
addEdge(adj, 7, 12, 312)
addEdge(adj, 7, 8, 253)
addEdge(adj, 7, 10, 254)

problem = Problem(0, 12, adj)
result = A_star(problem)

if result is not None:
    path = get_best_path(result)
    print("Best path :")
```

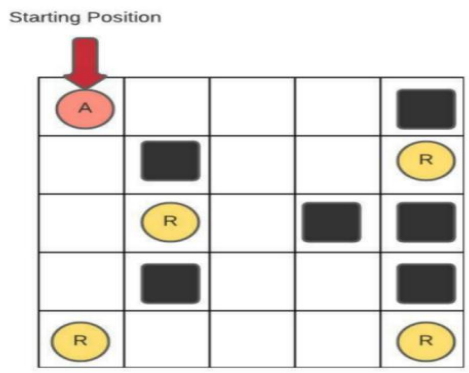
```
        for i in path:
            print(city_map[i],sep=" ")


        path_cost=cost(path,adj)
        print(path_cost)
    else:
        print("No path found")
main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\VIVEK\Desktop\AI_ Lab> python -u "c:\Users\VIVEK\Desktop\AI_ Lab\Lab Assignment 6\que1a.py"
Cities Visited 9
Best path :
Chicago
Cleveland
Buffalo
Syracuse
Boston
996
PS C:\Users\VIVEK\Desktop\AI_ Lab>
```

Q2 Consider a maze as a 5x5 matrix input where 0 denotes an empty tile, 1 denotes an obstruction/wall, 2 denotes the start state and 3 denotes the reward. Assume valid actions as L,R,U,D where L=move_left, R=move_right, U=move_up, D=move_down,. Use the A* algorithm on the resultant maze for your agent to reach all the rewards, and keep a record of the tiles visited on the way. Consider the rewards as goals. If all goals are completed, then one can come out of the maze. Assume heuristic cost and evaluation cost with proper justification.



ANS:

```
class PQ:
    def __init__(self):
        self.q=[]

    def isEmpty(self):
        return len(self.q)==0
    def right(self,i):
        return 2*i + 2
    def left(self,i):
        return 2*i + 1
    def parent(self,i):
        return (i-1)//2
    def insert(self, val):
        self.q.append(val)
        i = len(self.q) - 1

        while i>0:
            p=self.parent(i)
            if self.q[p].f > self.q[i].f:
                self.q[p], self.q[i] = self.q[i], self.q[p]
                i=p
            else:
                break

    def pop(self):
        if self.isEmpty():
            return None
        if len(self.q)==1:
            return self.q.pop()
        r=self.q[0]
        self.q[0]=self.q.pop()
```

```
        self.heapify(0)
        return r
    def heapify(self,i):
        l=self.left(i)
        r=self.right(i)
        mi=i
        if l<len(self.q) and self.q[l].f<self.q[mi].f:
            mi=l
        if r<len(self.q) and self.q[r].f<self.q[mi].f:
            mi=r
        if mi!=i:
            self.q[i],self.q[mi]=self.q[mi],self.q[i]
            self.heapify(mi)

def heuristic(pos, remaining):
        if len(remaining)==0:
            return 0
        x,y=pos
        return min(abs(x-rx)+abs(y-ry) for rx,ry in remaining)
class Node:

    def __init__(self,state,parent,action,g,remaining):
        self.state = state
        self.parent = parent
        self.action = action
        self.g = g
        self.remaining = remaining
        self.h = heuristic(state,remaining)
        self.f = self.g + self.h

    def __lt__(self,other):
```

```
        return (self.f,self.g) < (other.f,other.g)

class MazeProblem:

    def __init__(self,grid):
        self.grid=grid
        self.n=len(grid)
        self.start=None
        self.rewards=set()

        for i in range(self.n):
            for j in range(self.n):
                if grid[i][j]==2:
                    self.start=(i,j)
                if grid[i][j]==3:
                    self.rewards.add((i,j))

    def INITIAL(self):
        return self.start,self.rewards

    def Is_Goal(self,state):
        pos,remaining=state
        return len(remaining)==0

    def ACTIONS(self,state):
        x,y=state[0]
        acts=[]
        for dx,dy,a in [(1,0,'D'),(-1,0,'U'),(0,1,'R'),(0,-1,'L')]:
            nx,ny=x+dx,y+dy
            if 0<=nx<self.n and 0<=ny<self.n and self.grid[nx][ny]!=1:
                acts.append((nx,ny,a))
```

```
        return acts

    def RESULT(self,state,action):
        (nx,ny,a)=action
        pos,remaining=state
        new_remaining=set(remaining)
        if (nx,ny) in new_remaining:
            new_remaining.remove((nx,ny))
        return ( (nx,ny), frozenset(new_remaining) )

    def ACTION_COST(self,s,S):
        return 1
def Expand(problem,node):

    children=[]
    state=(node.state,node.remaining)

    for action in problem.ACTIONS(state):

        new_state,new_remaining = problem.RESULT(state,action)
        child_g=node.g+1

        child=Node(
            state=new_state,
            parent=node,
            action=action[2],
            g=child_g,
            remaining=new_remaining
        )

        children.append(child)
```

```
    return children
def A_star(problem):

    frontier=PQ()

    start_pos,start_rem=problem.INITIAL()
    start=Node(start_pos,None,None,0,frozenset(start_rem))

    frontier.insert(start)
    reached={(start.state,start.remaining):start}

    while not frontier.isEmpty():

        node=frontier.pop()

        if problem.Is_Goal((node.state,node.remaining)):
            return node

        for child in Expand(problem,node):

            s=(child.state,child.remaining)

            if s not in reached or child.g<reached[s].g:
                reached[s]=child
                frontier.insert(child)

    return None
def get_path(node):
    p=[]
    while node:
```

```
        p.append(node.state)
        node=node.parent
    return p[::-1]

maze=[
[2,0,0,0,1],
[0,1,0,0,3],
[0,3,0,1,1],
[0,1,0,0,1],
[3,0,0,0,3]
]

problem=MazeProblem(maze)
result=A_star(problem)

if result:
    p=get_path(result)
    print("Path:")
    for i in p:
        print(i)
    print("Steps:",len(p)-1)
else:
    print("No Solution")
```

```
PS C:\Users\VIVEK\Desktop\AI_ Lab> python -u "c:\Users\VIVEK\Desktop\AI_ Lab\Lab Assignment 6\q2.py"
Path:
(0, 0)
(0, 1)
(0, 2)
(1, 2)
(1, 3)
(1, 4)
(1, 3)
(1, 2)
(2, 2)
(2, 1)
(2, 0)
(3, 0)
(4, 0)
(4, 1)
(4, 2)
(4, 3)
(4, 4)
Steps: 16
PS C:\Users\VIVEK\Desktop\AI_ Lab>
```